Pashov Audit Group

Spicenet Security Review

January 12th 2026 - January 17th 2026

Contents

1. About Pashov Audit Group 4
2. Disclaimer 4
3. Risk Classification 4
4. About Spicenet 5
5. Executive Summary 5
6. Findings 6

High findings 9

[H-01] Replay attack via unvalidated `ChainBatch.hash` 9
[H-02] Non-7702 on-demand mode accepts any tx hash and triggers solver payouts 9
[H-03] Spice deposit verification allows spoofed deposits from untrusted contracts 11
[H-04] Unauthenticated 7702 actions can drain solver-held funds 13
[H-05] Withdrawals bypass on-chain verification 16

Medium findings 18

[M-01] No upper bound on query limit parameter 18
[M-02] On-demand execute updates balances without event verification 18
[M-03] Balance map overwrites multiple tokens per chain 19
[M-04] Delegate authorization can be overwritten by any user 20
[M-05] Deposit verification logic flaw 21
[M-06] Integer precision loss in amount verification 22
[M-07] Lack of input constraints leads to database resource exhaustion 23
[M-08] Post `/venue-tx-hashes` lacks transaction authenticity check 23
[M-09] Non-7702 on-demand execution vulnerable to replay attacks 24
[M-10] Solver gas can be exhausted via unprotected action execution endpoint 25
[M-11] `executeChainAuthorizations` fails on native token transfer in batch 25
[M-12] Lack of address validation in `/spicedeposit` API 26

Low findings 27

[L-01] Nonce race condition in concurrent authorization signing 27
[L-02] Negative balance allowed on withdrawal 27
[L-03] Missing `signatureType` in intent hash 28
[L-04] Fetch and verify `fetchAndVerifyDepositEvents` has incorrect nested loop logic 28
[L-05] Intent expiry not enforced at API layer 29
[L-06] Error message information disclosure 30
[L-07] Missing rate limiting enables denial of service 31
[L-08] Missing helmet middleware for security headers 31
[L-09] Exposing application framework with default `X-Powered-By` header 33
[L-10] Sensitive operational data is logged verbatim 33
[L-11] Sensitive user data and signatures are logged to stdout 34
[L-12] Hardcoded RPC URLs embed API keys 34
[L-13] Airdrop faucet missing rate limits 35
[L-14] Race condition in `intentIndex/steps/:stepIn` 36
[L-15] Missing transaction simulation causes unnecessary gas loss 36
[L-16] Execute `executeChainAuthorizations` performs unsafe address equality check 37
[L-17] Apis in `app.ts` accept arbitrary input without format check 38
[L-18] Get `/venue-tx-hashes/verify` fails when optional parameters are omitted 38
[L-19] Missing address normalization in `/spicedeposit` API 39
[L-20] Legacy intent APIs should be removed to avoid inconsistency 39

[L-21] Single solver address creates transaction bottleneck and DoS risk 40
[L-22] Unreliable cross-chain balance aggregation in `/spicedeposit/:address` 40

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Spicenet

Spicenet is a cross-chain transaction execution API that processes EIP-7702 account abstraction intents and facilitates multi-chain token transfers through a solver-based architecture. The system enables users to submit signed transaction batches that are executed across multiple blockchain networks including Ethereum testnets, Arbitrum, Base, and other Layer 1/Layer 2 chains while maintaining balance tracking and transaction state management.

5. Executive Summary

A time-boxed security review of the **pepper-research/tx-submission-api** and **pepper-research/solver-evm-poc** repositories was done by Pashov Audit Group, during which **defsec**, **Bloqarl**, **Shurikenzer**, **Rayaa** engaged to review **Spicenet**. A total of **39** issues were uncovered.

Protocol Summary	
Project Name	Spicenet
Protocol Type	Cross-chain Transactions
Timeline	January 12th 2026 - January 17th 2026

Review commit hashes:

- f9f8544c11144c204d2df018365641dc73ab2d2c (pepper-research/tx-submission-api)
- 63e69bf75c98f7d0ba47f38d70ced075bbaeb71e (pepper-research/solver-evm-poc)

Fixes review commit hashes:

- 4d9f75eda15f5bd89df8d77ad4fb77d439917d31 (pepper-research/tx-submission-api)
- 2b0d76d2b821a6bdfdefe822092ba9a71bdc8c6f (pepper-research/solver-evm-poc)

Scope

`abi.ts` `app.ts` `authorization.ts` `chains.ts` `crest.ts` `index.ts` `step.ts` `types.ts` `verify.ts` `schema.ts` `store.ts` `airdrop.ts` `viem.ts` `Delegate.sol`

6. Findings

Findings count

Severity	Amount
High	5
Medium	12
Low	22
Total findings	**39**

Summary of findings

ID	Title	Severity	Status
[H-01]	Replay attack via unvalidated `ChainBatch.hash`	High	**Resolved**
[H-02]	Non-7702 on-demand mode accepts any tx hash and triggers solver payouts	High	Acknowledged
[H-03]	Spice deposit verification allows spoofed deposits from untrusted contracts	High	Acknowledged
[H-04]	Unauthenticated 7702 actions can drain solver-held funds	High	Acknowledged
[H-05]	Withdrawals bypass on-chain verification	High	Acknowledged
[M-01]	No upper bound on query limit parameter	Medium	**Resolved**
[M-02]	On-demand execute updates balances without event verification	Medium	Acknowledged
[M-03]	Balance map overwrites multiple tokens per chain	Medium	**Resolved**
[M-04]	Delegate authorization can be overwritten by any user	Medium	**Resolved**
[M-05]	Deposit verification logic flaw	Medium	Acknowledged
[M-06]	Integer precision loss in amount verification	Medium	**Resolved**
[M-07]	Lack of input constraints leads to database resource exhaustion	Medium	**Resolved**
[M-08]	Post `/venue-tx-hashes` lacks transaction authenticity check	Medium	Acknowledged

ID	Title	Severity	Status
[M-09]	Non-7702 on-demand execution vulnerable to replay attacks	Medium	Acknowledged
[M-10]	Solver gas can be exhausted via unprotected action execution endpoint	Medium	Acknowledged
[M-11]	`executeChainAuthorizations` fails on native token transfer in batch	Medium	**Resolved**
[M-12]	Lack of address validation in `/spicedeposit` API	Medium	Acknowledged
[L-01]	Nonce race condition in concurrent authorization signing	Low	**Resolved**
[L-02]	Negative balance allowed on withdrawal	Low	Acknowledged
[L-03]	Missing `signatureType` in intent hash	Low	Acknowledged
[L-04]	Fetch and verify `fetchAndVerifyDepositEvents` has incorrect nested loop logic	Low	Acknowledged
[L-05]	Intent expiry not enforced at API layer	Low	Acknowledged
[L-06]	Error message information disclosure	Low	Acknowledged
[L-07]	Missing rate limiting enables denial of service	Low	**Resolved**
[L-08]	Missing helmet middleware for security headers	Low	**Resolved**
[L-09]	Exposing application framework with default `X-Powered-By` header	Low	**Resolved**
[L-10]	Sensitive operational data is logged verbatim	Low	Acknowledged
[L-11]	Sensitive user data and signatures are logged to stdout	Low	**Resolved**
[L-12]	Hardcoded RPC URLs embed API keys	Low	Acknowledged
[L-13]	Airdrop faucet missing rate limits	Low	**Resolved**
[L-14]	Race condition in `intentIndex/steps/:stepIn`	Low	Acknowledged
[L-15]	Missing transaction simulation causes unnecessary gas loss	Low	Acknowledged
[L-16]	Execute `executeChainAuthorizations` performs unsafe address equality check	Low	**Resolved**

ID	Title	Severity	Status
[L-17]	Apis in `app.ts` accept arbitrary input without format check	Low	Acknowledged
[L-18]	Get `/venue-tx-hashes/verify` fails when optional parameters are omitted	Low	**Resolved**
[L-19]	Missing address normalization in `/spicedeposit` API	Low	**Resolved**
[L-20]	Legacy intent APIs should be removed to avoid inconsistency	Low	Acknowledged
[L-21]	Single solver address creates transaction bottleneck and DoS risk	Low	Acknowledged
[L-22]	Unreliable cross-chain balance aggregation in `/spicedeposit/:address`	Low	Acknowledged

High findings

[H-01] Replay attack via unvalidated `ChainBatch.hash`

Severity

Impact: High

Likelihood: Medium

Description

The `ChainBatch.hash` field is user-provided but never validated against the actual computed hash of calls. The signed digest and replay protection use different values:

https://github.com/pepper-research/solver-evm-poc/blob/63e69bf75c98f7d0ba47f38d70ced075bbaeb71e/src/Delegate.sol#L132

```
// Line 132 - computeIntentHash() computes hash from actual calls
hashes[i] = keccak256(abi.encode(c.chainId, c.calls));

// Line 120 - execute() uses user-provided c.hash for replay protection
_registerChainBatch(intent.signature, c.hash);
```

An attacker can replay the same signed intent unlimited times by providing different `c.hash` values while keeping identical calls. Each submission creates a unique `chainBatchId`, completely bypassing replay protection.

Recommendations

Compute the hash from calls and use the computed value for replay protection:

```
// In execute(), replace line 120 with:
bytes32 computedHash = keccak256(abi.encode(c.chainId, c.calls));
_registerChainBatch(intent.signature, computedHash);
```

[H-02] Non-7702 on-demand mode accepts any tx hash and triggers solver payouts

Severity

Impact: High

Likelihood: Medium

Description

The `non-7702:on-demand` execution path allows users to submit a transaction hash as proof that they executed an on-chain action. Upon verification, the service triggers solver-funded transfers on destination chains. However, the verification only checks that the transaction receipt exists and has `status === "success"`. It does not validate that:

- The transaction `from` address matches the action's `user`.
- The transaction `to` and `input` match the expected call stored in the database.
- Event logs (e.g., ERC20 `Transfer`, bridge events) prove the intended action occurred.

This allows an attacker to submit any unrelated successful transaction hash and trigger solver-funded transfers.

Affected Code

src/routes/actions/index.ts

```
const receipt = await publicClient.getTransactionReceipt({ hash: txHash });
if (receipt.status !== "success") {
  res.status(400).json({ error: "Transaction failed or pending" });
  return;
}
```

After this minimal check, solver transfers are executed unconditionally:

src/routes/actions/index.ts#L361

```
await executeTokenTransfers(intentRecord.id, stepIndex);
The executeTokenTransfers(...) function sends funds from the solver account for any transfer
where sender equals the solver address:
```

src/step.ts#L364-L366

```
if (transfer.sender.toLowerCase() !== solverAccount.address.toLowerCase()) {
  continue;
}
// ... proceeds to send native/ERC20 transfer from solverAccount
```

Additionally, the handler auto-executes all remaining steps after a single verification:

src/routes/actions/index.ts#L380-L392

```
for (const remainingStep of allSteps) {
  if (remainingStep.status === "success" || remainingStep.index <= stepIndex) continue;
  await executeTokenTransfers(intentRecord.id, remainingStep.index);
  // ...
}
```

Impact

An attacker can drain solver funds by creating an action with tokenTransfers where from: "solver" routes funds to an attacker-controlled address, then "proving" it with any unrelated successful transaction hash. The auto-execution of remaining steps amplifies losses across multiple chains.

Recommendation

Before executing solver transfers:

- Verify tx.from === user.
- Verify tx.to and tx.input match the expected call stored in DB.
- Validate relevant event logs (ERC20 Transfer, bridge events).
- Execute only the verified step, not remaining steps automatically.

[H-03] Spice deposit verification allows spoofed deposits from untrusted contracts

Severity

Impact: High

Likelihood: Medium

Description

The `POST /spicedeposit` endpoint is intended to record user deposits by verifying that an on-chain transaction emitted a `Deposit(sender, token, amount)` event matching the request parameters.

However, `verifySpiceDeposit(...)` decodes `Deposit` logs from the **entire transaction receipt** and validates only the decoded event arguments `(sender, token, amount)`. It does not validate the emitting contract `address (log.address)` against any allowlist of trusted "Spice deposit" contracts.

On EVM chains, any contract can emit an event with the **same signature**. Therefore, an attacker can deploy or call a malicious contract that emits a `Deposit` event with attacker-chosen values and have the backend accept it as a valid deposit, even if no real deposit occurred in any official deposit contract.

Affected Code

`verifySpiceDeposit(...)` collects all `Deposit` events, but never checks `log.address` (the contract that emitted them):

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/verify.ts#L137-L175

The `/spicedeposit` handler trusts the boolean and credits balances:

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/app.ts#L306-L333

Impact

An attacker can credit internal balances without making a real deposit into any official deposit contract by emitting a spoofed `Deposit` event from an arbitrary contract. This breaks accounting integrity, as any downstream logic that relies on balances (eligibility, credit limits, payouts, routing, etc.) can be bypassed or manipulated.

Proof of Concept

Deploy a contract that emits an identical `Deposit` event:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract FakeDepositEmitter {
  event Deposit(address indexed sender, address indexed token, uint256 amount);

  function emitDeposit(address sender, address token, uint256 amount) external {
    emit Deposit(sender, token, amount);
  }
}
```

1. Deploy FakeDepositEmitter on a supported chain.
2. Call emitDeposit(attacker, tokenAddress, amount) and obtain the transaction hash txHash.
3. Call the API with matching parameters:

```
curl -sS -X POST "http://<host>:<port>/spicedeposit" \

  -H "content-type: application/json" \
  -d '{
    "user":"0xattacker...",
    "sender":"0xattacker...",
    "txHash":"0x<txHash_of_emitDeposit_call>",
    "chainId":84532,
    "tokenAddress":"0xdeadbeef...",
    "amount":"1000000000000000000",
    "isDeposit":true
  }'
```

Expected result: `verifySpiceDeposit(...)` decodes the spoofed `Deposit` log, sees matching `(sender, token, amount)`, and the API credits the attacker's balance even though no "real" deposit occurred.

Recommendation

- Maintain an allowlist of trusted deposit contract address(es) per chainId, and only accept Deposit events where log.address is in that allowlist.
- Require at least one matching Deposit event from a trusted emitter and reject receipts that contain zero events.
- Verify receipt.status === "success".
- Compare amounts as bigint (avoid Number(...) truncation/overflow).
- Prefer verifying the **actual asset movement** (e.g., ERC20 Transfer into the trusted deposit contract) instead of trusting a custom event alone.

[H-04] Unauthenticated 7702 actions can drain solver-held funds

Severity

Impact: High

Likelihood: Medium

Description

The API exposes endpoints that let any caller create an `action/intent` and then trigger its execution. Execution is performed by the server using `SOLVER_PRIVATE_KEY` to submit an EIP-7702 transaction that calls `Delegate.selfExecute(...)` from the solver account.

However, the action schema also lets clients specify token transfers, including transfers "from solver." The server persists those user-provided rows and later converts them into real on-chain ERC20/native transfers from the solver account before invoking the user's `Delegate.selfExecute(...)`.

Since `Delegate.execute(...)` allows "no-op" batches `(calls.length == 0)`, an attacker can sign a valid intent for their own EOA while providing empty calls. The intent still passes signature and time-window checks, while the solver-funded transfers execute in the same `selfExecute(...)` transaction.

Affected Code

Client-supplied token transfers can be persisted as solver-funded by mapping `from: "solver"` to the real solver address:

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/routes/actions/index.ts#L80-L105

```
// Store token transfers for this chain batch
for (let k = 0; k < batch.tokenTransfers.length; k++) {
  const transfer = batch.tokenTransfers[k]!;
  const sender = transfer.from === "solver" ? solverAddress : transfer.from;
  // @audit Untrusted client can force sender = solverAddress
  await tx.insert(tokenTransfers).values({
    // ...
    sender,
    receiver: transfer.to,
  });
}
```

Those rows are later executed as real on-chain transfers from the solver account, and the solver always appends a call into the user's `Delegate.selfExecute(...)`:

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/step.ts#L153-L218

```
const solverCalls = [];

for (const token_transfer of batchTokenTransfers) {
  // @audit Any row with sender == solverAccount.address becomes a solver-funded transfer
  if (token_transfer.sender !== solverAccount.address) { continue; }

  if (token_transfer.tokenAddress !== "0x0") {
    solverCalls.push({
      // ...
      to: token_transfer.tokenAddress as `0x${string}`,
      data: encodeFunctionData({
        abi: erc20Abi,
        functionName: "transfer",
        args: [
          token_transfer.receiver as `0x${string}`,
          token_transfer.tokenAmount,
        ],
      }),
      value: 0n,
    });
  } else {
    solverCalls.push({
      to: user as `0x${string}`,
      data: "0x" as const,
      value: token_transfer.tokenAmount ? token_transfer.tokenAmount : 0n,
    });
  }
}

solverCalls.push({
  // @audit Delegate.execute(...) is called even if all batches have empty calls (no-op intent)
  to: user as `0x${string}`,
  data: encodeFunctionData({
    abi: DELEGATE_ABI,
    functionName: "execute",
    args: [/* intent */],
  }),
  value: 0n,
```

```
});

await walletClient.writeContract({
  address: solverAccount.address as `0x${string}`,
  functionName: "selfExecute",
  args: [solverCalls],
});
```

On-chain, the delegate explicitly skips batches with empty calls, allowing a valid no-op intent:

- Delegate.sol:execute

```
// only check batches that were asked to be executed
if (c.calls.length == 0) continue; // @audit allows no-op intents

_registerChainBatch(intent.signature, c.hash);
_executeBatch(c.calls);
```

Impact

This allows an attacker to drain any ERC20/native balances held by the solver account by repeatedly creating actions that include "from solver" transfers and then triggering execution. The solver also pays gas for the drain transactions, enabling economic griefing even when balances are low.

Proof of Concept

1. Choose an EOA you control as user (attacker address).
2. Craft a 7702 intent where all chainBatches[*].calls are empty ([]), set a valid nbf/exp window, and sign the intent hash as expected by Delegate.execute(...).
3. Call POST /actions with: user = chainAuthorizations = [...] (valid EIP-7702 authorization for the attacker on the target chain)
   intents[0].chainBatches[0].tokenTransfers = [{ from: "solver", to: , token: , amount: }]
   intents[0].chainBatches[0].calls = []
4. Trigger execution via POST /actions/:actionId/intents/0/steps/0 (or the equivalent intent/step execution route).

Result: the solver submits a `selfExecute(...)` transaction that includes the solver → attacker transfer(s) and a no-op `execute(...)` call that still passes signature checks.

Recommendation

- Require authentication/authorization on any endpoint that can create actions/intents or trigger execution (e.g., service-to-service authentication or signed API keys + strict rate limits).
- Treat all solver-funded transfers as server-authorized only: Reject transfers from "solver" from untrusted clients, or require a separate solver-signed quote/commitment authorizing exact payouts.
- Add server-side validation that 7702 steps include the expected non-empty user calls and correspond to a validated business event, rather than accepting no-op intents.

[H-05] Withdrawals bypass on-chain verification

Severity

Impact: High

Likelihood: Medium

Description

The `/spicedeposit` endpoint handles deposits and withdrawals via an `isDeposit` flag. Deposits are verified on-chain via `verifySpiceDeposit()`, but when `isDeposit=false`, all on-chain verification is skipped. The code contains a TODO acknowledging this gap. The balance is directly modified without proof that a withdrawal transaction occurred, allowing attackers to drain balances with any arbitrary transaction hash.

Affected Code

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/app.ts#L306-L317
- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/app.ts#L328-L332

```
// @audit Withdrawals skip on-chain verification
if (isDeposit) {
  const verified = await verifySpiceDeposit({ txHash, sender, tokenAddress, chainId, amount,
isDeposit });
  if (!verified) {
    res.status(400).json({
      success: false,
      error: { message: "Spice deposit verification failed" },
    } as SpiceDepositResponse);
    return;
  }
}
// TODO: add on-chain verification for withdrawals once the exact
// withdrawal event name/signature is confirmed.
```

```
// @audit Balance modification without verification for withdrawals
set: {
  amount: isDeposit
    ? sql`${balances.amount} + ${amount}`
    : sql`${balances.amount} - ${amount}`, // No verification when isDeposit=false
},
```

Impact

An attacker can drain any user's balance to zero by calling the endpoint with `isDeposit=false` and any arbitrary transaction hash, without proof of on-chain withdrawal. This enables complete theft of user funds and, if exploited at scale, leads to protocol insolvency as all balances can be systematically drained.

Recommendations

1. Implement on-chain verification for withdrawals.
2. Alternatively, require a cryptographic signature from the user proving they authorized the withdrawal.

Medium findings

[M-01] No upper bound on query limit parameter

Severity

Impact: Medium

Likelihood: Medium

Description

In [src/app.ts:542-546](), the `count` query parameter has no upper bound:

```
let limitCount = count ? parseInt(count as string, 10) : 1;

if (limitCount > 0) {
  queryOptions.limit = limitCount;
}

const results = await db.query.venueTxHashes.findMany(queryOptions);
```

An attacker can request `?count=10000000` to:

1. Cause database to return massive result sets.
2. Exhaust server memory.
3. Create denial of service condition.
4. Slow down database for other users.

Recommendation

Set a maximum limit.

[M-02] On-demand execute updates balances without event verification

Severity

Impact: Medium

Likelihood: Medium

Description

In `src/routes/actions/index.ts:337-359`, the "execute" action for non-7702: on-demand mode updates balances based on stored transfer data rather than verifying actual on-chain events:

```
} else {
  // execute: update balances + execute solver transfers
  const stepTransfers = await db.query.tokenTransfers.findMany({
    where: and(
      eq(tokenTransfers.intentId, intentRecord.id),
      eq(tokenTransfers.chainAuthorizationIndex, stepIndex)
    ),
  });

  for (const transfer of stepTransfers) {
    if (transfer.sender.toLowerCase() === "solver") continue;

    // Balance updated based on stored data, not verified on-chain!
    await db.insert(balances)
      .values({
        user: user,
        chainId: BigInt(chainId),
        tokenAddress: transfer.tokenAddress,
        amount: transfer.tokenAmount.toString(),
      })
      .onConflictDoUpdate({
        target: [balances.user, balances.chainId, balances.tokenAddress],
        set: { amount: sql`${balances.amount} + ${transfer.tokenAmount.toString()}` },
      });
  }
```

The code only verifies `receipt.status === "success"` (line 307), but does not verify:

1. The transaction actually transferred the expected tokens.
2. The amounts match what was stored.
3. The sender and receiver match.

Recommendations

1. Decode and verify the transaction's Transfer events.
2. Compare event data against stored token transfer expectations.
3. Only credit balances that match verified on-chain events.

[M-03] Balance map overwrites multiple tokens per chain

Severity

Impact: Medium

Likelihood: Medium

Description

In `src/app.ts:383-388`, the token map construction has a bug that overwrites previous entries:

```
const tokenMap: Record<string, Record<string, string>> = {};
for (const balance of balancesRes) {
  // This overwrites the entire chain object each iteration!
  tokenMap[balance.chainId.toString()] = {
    [balance.tokenAddress]: balance.amount.toString(),
  };
}
```

If a user has multiple tokens on the same chain:

- Token A: 100
- Token B: 200

After the loop, only Token B will be in the map because each iteration creates a new object.

Expected behavior:

```
{ "84532": { "0xTokenA": "100", "0xTokenB": "200" } }
```

Actual behavior:

```
{ "84532": { "0xTokenB": "200" } }
```

Users see incomplete balance information.

Recommendations

```
const tokenMap: Record<string, Record<string, string>> = {};
for (const balance of balancesRes) {
  if (!tokenMap[balance.chainId.toString()]) {
    tokenMap[balance.chainId.toString()] = {};
  }
  tokenMap[balance.chainId.toString()][balance.tokenAddress] = balance.amount.toString();
}
```

[M-04] Delegate authorization can be overwritten by any user

Severity

Impact: Medium

Likelihood: Medium

Description

In `src/db/store.ts:129-137` and `src/routes/actions/index.ts:136-163`, delegate authorizations are stored with `onConflictDoUpdate`, allowing any user to overwrite another user's stored authorization:

```
// store.ts:130-137
for (const authValue of authorizationValues) {
  await tx.insert(delegateAuthorization).values(authValue).onConflictDoUpdate({
    target: [delegateAuthorization.user, delegateAuthorization.chainId],
    set: {
      authorization: authValue.authorization,  // Overwrites existing!
    }
  });
}
```

The `user` field comes directly from the request body without verification:

```
// store.ts:82
user: address,  // From req.body.address - not verified
```

Note: This does not allow executing transactions as the victim because EIP-7702 authorizations are cryptographically signed. The EVM verifies that `ecrecover(r, s, yParity)` matches the account being delegated. An attacker cannot forge a valid signature without the victim's private key.

Actual attack (DoS only):

1. Attacker submits a transaction with `address: "0xVICTIM"`.
2. Includes an invalid `authorization` array.
3. Victim's existing valid delegate authorization is overwritten with an invalid one.
4. Victim's pending executions fail because the stored authorization is now invalid.
5. Victim must re-submit their authorization to recover.

Recommendations

Verify that the caller owns the address through signature proof.

[M-05] Deposit verification logic flaw

Severity

Impact: Medium

Likelihood: Medium

Description

In `src/verify.ts:165-173`, the deposit verification logic has a flaw:

```
let verified = true;
for (const event of events) {
  if (event.sender !== sender ||
      event.token !== tokenAddress ||
      Number(event.amount) !== Number(amount)
  ) {
      verified = false;
      break;
  }
}
return verified;
```

This logic:

1. Returns `true` if the `events` array is empty (no deposits found).
2. Returns `true` only if ALL events match the criteria (when you likely want ANY event to match).
3. Fails to verify that the deposit actually exists if no matching events are found.

An attacker could submit a transaction hash that has no `Deposit` events (e.g., a failed transaction or a completely unrelated transaction), and the verification would return `true` because the loop never executes.

Recommendations

1. Return `false` if no events are found.
2. Change the logic to verify that at least one event matches the criteria.

[M-06] Integer precision loss in amount verification

Severity

Impact: Medium

Likelihood: Medium

Description

In [src/verify.ts](), `bigint` values are converted to `Number` for comparison:

```
// Line 88
Number(event.amount) !== Number(tokenTransfer.tokenAmount)

// Line 168
Number(event.amount) !== Number(amount)
```

JavaScript's `Number` type can only safely represent integers up to `2^53 - 1` (approximately 9 quadrillion). Token amounts in blockchain applications regularly exceed this limit, especially for tokens with 18 decimal places. For example:

- 1 million tokens with 18 decimals = `1e24` (exceeds safe integer range).
- This conversion causes precision loss and can lead to incorrect verification results.

Recommendations

Replace `Number()` comparisons with direct `bigint` comparisons.

[M-07] Lack of input constraints leads to database resource exhaustion

Severity

Impact: Medium

Likelihood: Medium

Description

Anyone can call the `POST /actions` API to create a new action, which inserts data into multiple database tables. However, this API does not enforce any limits on input, such as maximum array length or maximum string length.

As a result, a malicious user can repeatedly submit requests with the maximum allowed request body size, inserting excessive data into the database on each call. By spamming this endpoint, an attacker can significantly increase database load and storage usage, potentially leading to performance degradation or denial of service.

Recommendations

Since this API accepts large input payloads, it should strictly validate and limit input data. This includes enforcing maximum sizes for arrays, strings, and nested objects, as well as applying request rate limits.

[M-08] Post `/venue-tx-hashes` lacks transaction authenticity check

Severity

Impact: Medium

Likelihood: Medium

Description

The `POST /venue-tx-hashes` API can be called by anyone to record an on-chain transaction. However, it does not perform any validation to ensure the correctness or authenticity of the submitted data. As a result, fake or invalid transaction records can be stored in the database.

Recommendations

Verify the on-chain transaction data before storing it in the database.

[M-09] Non-7702 on-demand execution vulnerable to replay attacks

Severity

Impact: High

Likelihood: Medium

Description

In the `POST /actions/:actionId/intents/:intentIndex/steps/:stepIndex` API, when handling intents in `non-7702:on-demand` mode with the `execute` action type, the intent step status is updated to `success` only if `executeTokenTransfers()` completes without error. However, if `executeTokenTransfers()` throws an exception, the intent step status is not updated to `error`. As a result, the step remains in `created` status and can be retried indefinitely.

An attacker can exploit this behavior by intentionally crafting the last transfer in the transfer array to revert. This causes `executeTokenTransfers()` to throw, while leaving the intent step status unchanged, enabling a replay attack through repeated execution attempts.

Additionally, although the API attempts to prevent replay attacks using `request.body.hash`, this value is fully controlled by the caller. A malicious user can supply an arbitrary hash to bypass the replay protection entirely.

Recommendations

- Update the intent step status to `error` whenever `executeTokenTransfers()` throws an exception to prevent further execution attempts.
- Do not rely on a user-provided hash for replay protection.

[M-10] Solver gas can be exhausted via unprotected action execution endpoint

Severity

Impact: Medium

Likelihood: Medium

Description

Anyone can call the `POST /actions` API to create a new action and then call the `POST /actions/:actionId/intents/:intentIndex/steps/:stepIndex` API to execute a specific intent step at no cost.

However, under the hood, each execution requires the solver address to submit an on-chain transaction and pay the gas fee, while receiving no compensation in return. As a result, an attacker can repeatedly call these APIs to force the solver to spend native tokens.

This creates a gas-draining DoS vector, where an attacker can exhaust the solver's balance, preventing legitimate users from using the service.

Recommendations

- Restrict gas sponsorship to whitelisted users.
- Introduce an incentive or fee mechanism to cover gas costs.

[M-11] `executeChainAuthorizations` fails on native token transfer in batch

Severity

Impact: Medium

Likelihood: Medium

Description

Transactions submitted via `executeChainAuthorizations()` always revert when a native token transfer is included in the token transfer batch.

Specifically, the logic determines whether a transfer is a native token transfer by comparing the token address to `"0x0"`. However, when creating a new `Intent`, the token address is validated and must be a valid address. As a result, the token address is never equal to `"0x0"`.

Consequently, native token transfers are incorrectly treated as ERC20 token transfers, causing the contract to attempt calling `transfer()` on an invalid token.

```
for (const token_transfer of batchTokenTransfers) {
  if (token_transfer.sender !== solverAccount.address) { continue; }
  if (token_transfer.tokenAddress !== "0x0") {
    solverCalls.push({
      to: token_transfer.tokenAddress as `0x${string}`,
      data: encodeFunctionData({
        abi: erc20Abi,
        functionName: "transfer",
        args: [
          token_transfer.receiver as `0x${string}`,
          token_transfer.tokenAmount,
        ],
      }),
      value: 0n,
    });
  } else {
    ...
  }
}
```

Recommendations

Update the condition to correctly detect native token transfers by comparing against the zero address:

```
if (token_transfer.tokenAddress !== "0x0000000000000000000000000000000000000000")
```

[M-12] Lack of address validation in `/spicedeposit` API

Severity

Impact: Medium

Likelihood: Medium

Description

The API endpoint `POST /spicedeposit` can be called by anyone and does not verify the user address provided in the request body. As a result, an attacker can submit an arbitrary user address, causing the balance of that address to be updated without any corresponding on-chain transaction. This leads to incorrect and inconsistent balance data.

Recommendations

This API should only be accessible to authenticated users. The user address must be derived from a trusted source, such as the JWT access token, rather than being supplied directly by the caller in the request body.

Low findings

[L-01] Nonce race condition in concurrent authorization signing

Description

In [src/step.ts:147-150](), the solver's authorization nonce is calculated without locking:

```
authList.push(
  await solverAccount.signAuthorization({
    address: chains.get(Number(auth.chainId))
      ?.delegateContractAddress as `0x${string}`,
    chainId: chains.get(Number(auth.chainId))?.chainId || 0,
    nonce: (await publicClient.getTransactionCount(solverAccount)) + 1,
  })
)
```

If two `executeChainAuthorizations` calls run concurrently for different intents:

1. Both call `getTransactionCount()` and get the same value N.
2. Both calculate nonce as N + 1.
3. Both sign authorizations with the same nonce.
4. One transaction may fail or authorizations may conflict.

Additionally, `getTransactionCount()` returns confirmed transactions, not pending, so the calculated nonce may already be in use by a pending transaction.

Recommendation

1. Use a nonce manager with locking/atomic increment.
2. Or let viem handle nonce automatically when possible.
3. Add retry logic for nonce-related failures.

[L-02] Negative balance allowed on withdrawal

Description

In `src/app.ts:329-332`, withdrawal operations can result in negative balances:

```
set: {
  amount: isDeposit
    ? sql`${balances.amount} + ${amount}`
    : sql`${balances.amount} - ${amount}`,
},
```

The database schema at `src/db/schema.ts:202-215` confirms that no check constraint exists:

```
export const balances = pgTable(
  "balances",
  {
    user: varchar("user").notNull(),
    chainId: bigint("chain_id", { mode: "bigint" }).notNull().references(() => chain.id),
    amount: numeric("amount").notNull(),  // No check constraint
    tokenAddress: varchar("token_address").notNull(),
  },
  // ...
)
```

Combined with the missing withdrawal verification (H-01), an attacker can:

1. Submit a withdrawal for more than their balance.
2. The balance becomes negative.
3. Accounting is corrupted between on-chain and off-chain states.

Recommendations

Add a check constraint at the database level.

[L-03] Missing `signatureType` in intent hash

Description

The `computeIntentHash` function excludes `signatureType` from the signed digest:

```
return keccak256(abi.encode(intent.nbf, intent.exp, hashes));
// signatureType not included
```

Currently low impact as only "ecdsa" is supported. However, if additional signature types are added in the future (e.g., EIP-1271, BLS), signatures could potentially be valid across different verification methods if both accept the same digest format.

Include `signatureType` in the hash for future proofing.

[L-04] Fetch and verify `fetchAndVerifyDepositEvents` has incorrect nested loop logic

Description

In [src/verify.ts](), the verification logic has a nested loop that does not correctly match transfers to events:

```
let verified = true;
for (const tokenTransfer of tokenTransfersBatch) {
  for (const event of events) {
      if (event.sender !== user ||
          event.sender !== tokenTransfer.sender ||
          event.token !== (tokenTransfer.tokenAddress === "0x0" ? zeroAddress :
tokenTransfer.tokenAddress) ||
          Number(event.amount) !== Number(tokenTransfer.tokenAmount)
      ) {
          verified = false;
          break;
      }
  }
  if (!verified) break;
}
```

Issues:

1. For each token transfer, it checks all events. If any event does not match, it sets `verified = false`. This is backwards; it should verify that at least one event matches each transfer.
2. If `events` is empty, the inner loop never runs; `verified` stays `true`.
3. The `break` only exits the inner loop, but then `if (!verified) break` exits the outer loop. This means the first non-matching event fails the entire verification.

Recommendations

```
let verified = true;
for (const tokenTransfer of tokenTransfersBatch) {
  const matchingEvent = events.find(event =>
    event.sender === user &&
    event.sender === tokenTransfer.sender &&
    event.token === (tokenTransfer.tokenAddress === "0x0" ? zeroAddress :
tokenTransfer.tokenAddress) &&
    event.amount === tokenTransfer.tokenAmount
  );

  if (!matchingEvent) {
    verified = false;
    break;
  }
}
```

[L-05] Intent expiry not enforced at API layer

Description

In `src/db/schema.ts:42-43`, intents have `nbf` (not before) and `exp` (expiry) fields:

```
nbf: bigint("nbf", { mode: "bigint" }),
exp: bigint("exp", { mode: "bigint" }),
```

The API layer in `src/step.ts` does not check these values before execution.

However, these values are passed to the delegate contract:

```
// step.ts:189-194
solverCalls.push({
  data: encodeFunctionData({
    abi: DELEGATE_ABI,
    functionName: "execute",
    args: [
      {
        nbf: intentRecord.nbf ?? 0n,  // ← Passed to contract
        exp: intentRecord.exp ?? 0n,  // ← Passed to contract
        // ...
      },
    ],
  }),
});
```

Recommendations

Add API-level validation for defense in depth.

[L-06] Error message information disclosure

Description

Throughout the codebase, error messages are returned to clients. For example, in [src/app.ts:162-165](src/app.ts:162-165):

```
res.status(500).json({
  success: false,
  error: {
    message: error instanceof Error ? error.message : "Internal server error",
  },
});
```

Note: This code only returns `error.message`, not `error.stack`, so full stack traces with file paths are not exposed.

However, error messages could still reveal:

1. Database schema details (e.g., "duplicate key value violates unique constraint on 'balances'")
2. Internal service connection information (e.g., "connect ECONNREFUSED 10.0.0.5:5432")
3. Table/column names from database errors

The actual risk is limited since only the message string is exposed, not full stack traces.

Recommendations

1. Sanitize database and connection error messages before returning to clients.
2. Use generic error messages for 500 errors.
3. Log detailed errors server-side for debugging.

[L-07] Missing rate limiting enables denial of service

Description

The Express.js application at `src/app.ts` has no rate limiting middleware configured. All endpoints are vulnerable to abuse:

```
// src/app.ts - No rate limiting on any route
app.use(cors());
app.use(express.json());

app.post("/transaction/submit", async (req, res) => { ... });
app.get("/intent/:intentId/step/:stepId/status", async (req, res) => { ... });
app.post("/spicedeposit", async (req, res) => { ... });
app.post("/rfqt", async (req, res) => { ... });
app.post("/venue-tx-hashes", async (req, res) => { ... });
app.post("/airdrop/:chainId/:tokenId", async (req, res) => { ... });
```

Recommendations

Implement rate limiting using `express-rate-limit`.

[L-08] Missing helmet middleware for security headers

Description

The application at `src/app.ts` does not use the Helmet middleware, which sets various HTTP security headers to protect against common web vulnerabilities.

```
// src/app.ts:83-95
export function createApp(): Application {
  const app = express();

  if (!process.env.SOLVER_PRIVATE_KEY) {
    throw new Error("SOLVER_PRIVATE_KEY environment variable is required");
  }

  app.use(cors());
  app.use(express.json());
  // No helmet() middleware
```

Missing security headers include:

The table outlines several important HTTP security headers that are missing from the application at `src/app.ts`. Each header serves a specific purpose in enhancing the security posture of web applications, and their absence can expose the application to various vulnerabilities.

The first header, `X-Content-Type-Options`, is designed to prevent MIME sniffing, which can lead to MIME confusion attacks. Without this header, browsers may incorrectly interpret the content type of a resource, potentially allowing malicious content to be executed.

Next, the `X-Frame-Options` header helps prevent clickjacking attacks by controlling whether a page can be displayed in a frame. If omitted, the application could be vulnerable to UI redress attacks, where users are tricked into interacting with a malicious site.

The `Strict-Transport-Security` header enforces the use of HTTPS, which is crucial for protecting data in transit. The absence of this header can lead to downgrade attacks, where an attacker forces a connection to an insecure HTTP version.

Another critical header is `X-XSS-Protection`, which activates the browser's built-in cross-site scripting (XSS) filter. Without this protection, the application is susceptible to reflected XSS attacks, where malicious scripts can be executed in the user's browser.

The `Content-Security-Policy` header controls resource loading, significantly mitigating risks associated with XSS and data injection attacks. Not having this header leaves the application exposed to various injection vulnerabilities.

Additionally, `X-DNS-Prefetch-Control` manages DNS prefetching, which can help prevent privacy leaks by controlling how browsers resolve domain names before a user navigates to them. Its absence may inadvertently expose user browsing patterns.

Finally, the `Referrer-Policy` header governs the information that is sent as the HTTP referrer. Without it, there is a risk of information leakage, where sensitive data could be unintentionally shared with third-party sites.

In summary, the lack of these security headers in the application can lead to significant vulnerabilities, highlighting the need for the implementation of the Helmet middleware to enhance security.

Recommendations

Install and configure Helmet:

```
import helmet from 'helmet';

export function createApp(): Application {
  const app = express();

  app.use(helmet());
  // ... rest of configuration
}
```

[L-09] Exposing application framework with default `X-Powered-By` header

Description

Express.js by default sends the `X-Powered-By: Express` header with every response. This header is present in the application at `src/app.ts` because it is not explicitly disabled.

```
export function createApp(): Application {
  const app = express();
  // X-Powered-By header is enabled by default
  // ...
}
```

Example response header:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json
```

Recommendations

Disable the header explicitly:

```
const app = express();
app.disable('x-powered-by');
```

Or use Helmet middleware, which disables it automatically:

```
import helmet from 'helmet';
app.use(helmet());
```

[L-10] Sensitive operational data is logged verbatim

Description

The application logs full incoming request bodies for transaction submission and execution flows. These request bodies contain security-sensitive operational data, including intent identifiers (signatures), on-chain addresses, authorization payloads, token transfer details, and raw calldata.

Impact:

- **Information disclosure**: Intent IDs, addresses, calldata, and authorization material are exposed beyond the minimum required audience.
- **Operational risk**: Log poisoning and excessive log volume can degrade monitoring and increase costs.

Vulnerable code:

```
console.log("Received transaction submission request:", req.body);
```

It is recommended to:

- Do not log raw request bodies.
- Replace with structured, minimal summaries (e.g., intent ID prefix, chain IDs, counts of calls/transfers).

[L-11] Sensitive user data and signatures are logged to stdout

Description

Some endpoints log full request bodies and decoded on-chain data directly to stdout. These logs may include addresses, signatures, and authorization data.

Affected Code

```
// src/app.ts
console.log("Received transaction submission request:", req.body);

// src/verify.ts
console.log(tokenTransfer);
console.log(event);
```

Impact

If logs are accessible or retained improperly, sensitive data such as signatures or authorization payloads may be exposed, increasing the risk of misuse.

Recommendation

Remove these logs in production or redact sensitive fields (for example, signatures, authorizations, and calldata). Prefer logging only high-level identifiers like intentId or txHash and structured error messages.

[L-12] Hardcoded RPC URLs embed API keys

Description

Some RPC URLs are hardcoded in src/chains.ts and include third-party provider API keys directly in the URL. Since the repository or build output can be read by others, these keys become public and can be reused without permission.

These RPC endpoints are used to fetch security-relevant on-chain data (such as transaction receipts and logs). If the exposed keys are abused or rate-limited, it may cause temporary failures in verification or execution flows. However, this does not allow data tampering and mainly impacts availability.

Affected Code

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/chains.ts#L50-L180
- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/verify.ts#L45-L48

Impact

An attacker could reuse the exposed keys to exhaust RPC limits or quotas, leading to degraded availability or temporary verification failures. No direct fund loss or state corruption is possible.

Recommendation

Move RPC URLs with API keys to environment variables and treat them as secrets. Avoid committing provider keys to source control.

[L-13] Airdrop faucet missing rate limits

Description

The /airdrop/:chainId/:tokenId endpoint sends 200 test tokens per request without backend rate limiting or request tracking. There are no checks to prevent the same wallet or IP from requesting tokens multiple times. A TODO comment in the code shows this was planned but not implemented.

As confirmed by the client, this endpoint was used only for a testnet campaign. The tokens were test tokens minted by the team, all requests came from a controlled frontend, and users were verified. The endpoint was not intended to be exposed publicly, and time constraints prevented adding database-backed limits.

Affected Code

- https://github.com/pepper-research/tx-submission-api/blob/31b6981e5c7e3b4bc60ba7b6d5087895ca4ed0db/src/routes/airdrop.ts#L159-L173

```
// @audit No rate limiting implemented: TODO comment acknowledges this gap
// TODO: implement rate limiting
// TODO: save (request wallet, chain, token, amount, tx hash, tx status, timestamp)

const walletClient = createWalletClient({
  chain: chainConfig.viemChain,
  transport: http(),
});

// @audit No checks prevent repeated requests from same wallet/IP
const txHash = await walletClient.writeContract({
  address: tokenChainConfig.address,
  account: faucet,
```

```
  abi: erc20Abi,
  functionName: "transfer",
  args: [walletAddress, airdropAmountRaw],
});
```

Impact

In the current setup, the impact is limited to possible faucet exhaustion during testing, which does not affect real funds or production systems.

Recommendation

If the endpoint is reused or exposed in the future, basic backend rate limiting or request tracking should be added.

[L-14] Race condition in `intentIndex/steps/:stepIn`

Description

A malicious user can call the `POST /actions/:actionId/intents/:intentIndex/steps/:stepIndex` API twice concurrently, causing inconsistent and incorrect data to be stored in the database.

This API is responsible for executing a specific intent step and only processes steps with the `created` status. However, the step status is updated only after the on-chain transaction is confirmed. This creates a race condition: if the same step is executed concurrently, both requests pass the initial status check.

In this scenario, the first request may successfully execute the transaction on-chain, while the second request reverts due to replay protection. When the second request fails, it updates the step status to `error`, overwriting the correct state set by the first successful execution.

Recommendations

Implement a locking mechanism to prevent concurrent execution of the same intent step.

[L-15] Missing transaction simulation causes unnecessary gas loss

Description

In the `executeChainAuthorizations()` and `executeTokenTransfers()` functions, transactions are submitted directly to the blockchain without being simulated beforehand.

According to the Viem documentation, `writeContract` does not validate whether a contract call will succeed and may submit transactions that revert. Without first calling `publicClient.simulateContract`, transactions that are guaranteed to fail are still broadcast, causing the solver address to unnecessarily spend gas.

https://viem.sh/docs/contract/writeContract:

"The `writeContract` internally sends a transaction – it does not validate if the contract write will succeed (the contract may throw an error). It is highly recommended to simulate the contract write with `simulateContract` before you execute it."

Recommendations

Simulate the transaction using `publicClient.simulateContract` before submitting it to the blockchain.

[L-16] Execute `executeChainAuthorizations` performs unsafe address equality check

Description

In the `executeChainAuthorizations()` function, several address comparisons are performed without normalizing addresses to lowercase; this can lead to unexpected logic errors.

```
// @audit Addresses are not lowercased before comparison
if (
  !userDelegateRes || userDelegateRes !=
chains.get(Number(auth.chainId))?.delegateContractAddress as `0x${string}`
) {
  authList.push(
    (await db.query.delegateAuthorization.findFirst({
      where: and(
        // @audit Addresses are not lowercased before comparison
        eq(delegateAuthorization.user, user),
        eq(delegateAuthorization.chainId, BigInt(auth.chainId)),
      ),
    }))?.authorization
  );
}
```

The authorized delegate address is compared directly against the actual `Delegate` contract address without lowercasing either value. If both addresses are identical but differ in letter casing (uppercase vs lowercase), the comparison will fail. As a result, an authorization that already exists may be added again to the authorization list, leading to a transaction failure. Additionally, when querying the `delegateAuthorization` table, the `user` address is not normalized to lowercase. This may cause the query to fail to retrieve existing authorization records.

A similar issue exists in the following logic:

```
for (const token_transfer of batchTokenTransfers) {
  // @audit Addresses are not lowercased before comparison
  if (token_transfer.sender !== solverAccount.address) { continue; }
  ...
}
```

Here, the sender address stored in the database is compared directly with the solver account address without normalization. If their casing differs, valid transfers may be incorrectly ignored.

Recommendations

Always normalize addresses (e.g., convert to lowercase) before storing them in the database and before performing comparisons.

[L-17] Apis in `app.ts` accept arbitrary input without format check

Description

All APIs defined in `app.ts` do not validate the format of input data. Specifically, they do not validate addresses, transaction hashes, or signature formats. As a result, inputs can be arbitrary strings, leading to invalidly formatted data being stored in the database.

Recommendations

Add input validation using Zod, similar to the validation applied in the APIs defined in the `routes` folder.

[L-18] Get `/venue-tx-hashes/verify` fails when optional parameters are omitted

Description

In the `GET /venue-tx-hashes/verify/:txHash` API, the query parameters `actionType` and `venue` are optional. However, when querying the database to find the corresponding venue transaction, the logic always includes conditions for these two fields.

As a result, if `actionType` or `venue` is not provided in the query string, the query still applies equality checks using undefined values, causing the venue transaction to not be found even when a matching record exists.

```
const result = await db.query.venueTxHashes.findFirst({
  where: and(
    eq(venueTxHashes.txHash, txHash),
    eq(venueTxHashes.actionType, expectedActionType),
```

```
    eq(venueTxHashes.venue, expectedVenue),
  ),
});
```

Recommendations

Update the query logic to conditionally include `actionType` and `venue` in the `where` clause only when the corresponding query parameters are provided.

[L-19] Missing address normalization in `/spicedeposit` API

Description

In the `POST /spicedeposit` API, when persisting data into the `balances` table, neither the user address nor the token address is normalized to lowercase. As a result, multiple records can be created for the same user and token pair if the addresses differ only by character casing (uppercase vs. lowercase), instead of correctly updating the existing record. This leads to inconsistent and incorrect balance data.

Recommendations

Normalize all account addresses (e.g., convert to lowercase) before storing them in the database and before using them in lookup or update operations.

[L-20] Legacy intent APIs should be removed to avoid inconsistency

Description

Two APIs, `POST /transaction/submit` and `GET /intent/:intentId/step/:stepId/status`, are deprecated and should be removed or made to return an error for the following reasons:

- `POST /transaction/submit` stores data in tables that overlap with the current official APIs but use different data formats and logic. This can lead to data inconsistency across tables. Additionally, this API is not compatible with the current version of the `Delegate` contract, making it unusable.
- `GET /intent/:intentId/step/:stepId/status` does not support non-EIP-7702 operations, which are supported by the current official API implementation.

As a result, keeping these deprecated APIs may cause inconsistent data states and unsupported behavior. Only the current official APIs should be retained to ensure correctness and maintainability.

Recommendations

Remove the two deprecated APIs and rely exclusively on the current official APIs.

[L-21] Single solver address creates transaction bottleneck and DoS risk

Description

The system relies on a single solver address to submit all on-chain transactions on behalf of all users. On EVM-compatible chains, transactions from the same address are processed sequentially by nonce. A transaction with a higher nonce cannot be executed until all prior transactions with lower nonces are mined.

As a result, this design introduces a significant bottleneck and operational risk. If any earlier transaction becomes pending (e.g., due to low gas fees or network congestion), all subsequent transactions from the solver address are blocked. Additionally, when multiple users trigger executions concurrently, the solver may submit a large number of transactions, further increasing the likelihood of congestion and failures.

This architecture is inefficient, does not scale well, and can lead to long execution delays or temporary denial of service for users.

Recommendations

- Introduce multiple solver addresses instead of relying on a single solver, with proper coordination to handle pending or sped-up transactions.
- Use a third-party transaction relayer service.

[L-22] Unreliable cross-chain balance aggregation in `/spicedeposit/:address`

Description

The `GET /spicedeposit/:address` API is intended to return all token balances of a user across multiple chains. However, the service does not implement any mechanism to synchronize user balances with on-chain activity when users transfer or receive tokens.

As a result, the returned balances may quickly become outdated and inaccurate, diverging from the actual on-chain state. This makes the API unreliable and unusable for balance queries.

Recommendations

Replace this balance-tracking mechanism with a reliable third-party indexing service (e.g., on-chain indexers or blockchain data providers) that can accurately reflect real-time or near real-time on-chain balances.